Journal of Logical and Algebraic Methods in Programming 121 (2021) 100664

Contents lists available at ScienceDirect

Journal of Logical and Algebraic Methods in Programming

www.elsevier.com/locate/jlamp

Scalable optimal deployment in the cloud of component-based applications using optimization modulo theory, mathematical programming and symmetry breaking ☆

Mădălina Eraşcu *, Flavia Micota, Daniela Zaharie

Faculty of Mathematics and Informatics, West University of Timişoara, Romania

ARTICLE INFO

Article history:
Received 5 April 2020
Received in revised form 23 December 2020
Accepted 3 February 2021
Available online 10 February 2021

Keywords:
Cloud computing
Resource provisioning
Optimization modulo theory
Mathematical programming
Symmetry breaking

ABSTRACT

Automated deployment of *component-based applications* in the Cloud consists in the allocation of virtual machines (VMs) offers from various Cloud Providers such that the constraints induced by the interactions between components and by the components hardware/software requirements are satisfied and the performance objectives are optimized (e.g. costs are minimized). It can be formulated as a constraint optimization problem, hence, in principle, the optimization can be carried out automatically. In the case the set of VM offers is large (several hundreds), the computational requirement is huge, making the automatic optimization practically impossible with the current general optimization modulo theory (OMT) and mathematical programming (MP) tools. We overcame the difficulty by methodologically analyzing the particularities of the problem with the aim of identifying search space reduction methods. These are methods exploiting: *(i)* the *symmetries* of the general Cloud deployment problem, *(ii)* the *graph representation* associated to the structural constraints specific to each particular application, and *(iii)* their combination. An extensive experimental analysis has been conducted on four classes of real-world problems, using six symmetry breaking strategies and two types of optimization solvers.

As a result, the combination of a variable reduction strategy with a column-wise symmetry breaker leads to a scalable deployment solution, when OMT is used to solve the resulting optimization problem.

© 2021 Elsevier Inc. All rights reserved.

1. Introduction

The increased digital market demand determined the application service providers to invest in platforms that allow *component-based software development* using converged Infrastructure-as-a-Service (IaaS) and Platform-as-a-Service (PaaS) (Amazon Web Services, Azure, Google Cloud). This led to the wide adoption of *component-based software development*. Since the applications are deployed and executed on Cloud, they should *elastically* adapt by dynamically leasing or releasing resources and at the same time maintaining the expected level of Quality of Service (QoS). Additionally, component-based software development must ensure requirements like code reusability, short application creation and delivery time. All

☆ This work was partially supported by a grant of the Romanian National Authority for Scientific Research and Innovation, CNCS/CCCDI - UEFISCDI, project number PN-III-P2-2.1-PED-2016-0550, within PNCDI III.

* Corresponding author.
E-mail addresses: madalina.erascu@e-uvt.ro (M. Eraşcu), flavia.micota@e-uvt.ro (F. Micota), daniela.zaharie@e-uvt.ro (D. Zaharie).

https://doi.org/10.1016/j.jlamp.2021.100664
2352-2208/© 2021 Elsevier Inc. All rights reserved.

these led to new software engineering practices, such as DevOps, combining software development (Dev) and IT operations (Ops) with the aim of shortening the systems development life cycle by facilitating continuous delivery/deployment/integration [1]. DevOps methodology was already applied in, for example, the DICE[1] and MUSA[2] projects. The DICE toolchain includes *application modeling* at different abstraction levels using UML Profile which is used for deployment and optimal application configuration. The MUSA SecDevOps (Security DevOps) framework combines *security-by-design* mechanisms and continuous security assurance for systematic approach to cyber risks in the Multi-Cloud. More recently, SpecOps (Specification Operations) was used to make Microsoft Azure more reliable, by infusing *formal methods* in the DevOps cycle [2].

Application modeling, SecDevOps and SpecOps are used at the early stages of software development, hence they would benefit from techniques for the *automatic and optimal deployment*, as it: *(i)* would automatically synthesize deployment plans that are optimal by construction, and *(ii)* could integrate such deployment plans in the application modeling, in such a way that formal reasoning is possible on a model of the deployed application.

Typically, *automated deployment* involves the following steps: *(1)* selection of the computing resources, *(2)* the distribution/assignment of the application components over the available computing resources and *(3)* its dynamic modification to cope with peaks of user requests. Deployment is nowadays performed on a Cloud infrastructure which offers on-demand leasing model making the automated deployment problem substantially different from traditional resource allocation and scheduling problems encountered in grid computing, for example. This means that the number of available computing resources is potentially unlimited and appropriate deployment optimization strategies must be devised. To benefit from the multi-cloud approach and to optimize the adoption of Cloud resources from multiple Cloud Providers, the formulation of the deployment problem must explicitly take into account the on-demand leasing model for the allocation of Cloud resources and should satisfy the application-specific constraints. On the other hand, the overhead of acquiring and releasing the resources can be high, so it is important to organize the application components on the already acquired resources. Still, in order to benefit from the elasticity of the Cloud infrastructure and to cope with the users peaks, periodic redeployment can be performed.

In this paper, we tackle only the first two steps of the deployment problem. In particular, our approach is used to synthesize the initial static optimal deployment of the application which consists of an assignment of application components to VMs such that the application functional requirements are fulfilled and costs are minimized. We show that an integration between application modeling and automatic optimal deployment is actually possible as deployment information can be extracted from the application functional design (Section 2) and given as input to a solver that computes the optimal allocation of software components to computing nodes and synthesizes the corresponding deployment plans (Section 3). These deployment plans can then be used in Step 3 of the deployment or in various analysis tools (e.g. modular verification [3], cost analysis [4]). Differently to the initial static deployment, actions related to dynamic deployment are executed each time the application monitoring system detects changes and the Cloud engineer agrees scaling is necessary. This is typically due to a change on the number of instances of the services driven by the utilization patterns or on the resources associated to the virtual machines (VMs) used by the computing infrastructure. Existing scaling approaches exploit the so-called *horizontal scaling* [5] (i.e. components can locally increase/decrease their instance number depending on the values of monitored metrics) or *the vertical scaling* [6] (i.e. the possibility to dynamically add/remove computing resources). However, these assume that the overall application architecture remains unchanged and act on the components instance number or on the resources associated to the VMs used by the computing infrastructure. In reality, increasing/decreasing the number of components and resources might break the application functional requirements design which is fulfilled by the Cloud architecture synthesized though the initial deployment plan. Hence, redeployment plans should be taken into consideration. Since our approach to optimal deployment requires to know all the modifications at the architectural level (i.e. all the component instances that are expected to be deployed during a redeployment plan and the hardware/software characteristics of the computing resources that would possibly host the application), it is possible to compute optimal deployment strategies where instances of different components share resources. Such optimization might be impossible to achieve by solely relying on horizontal/vertical autoscaling [7]. As a consequence of the above, we formulate the optimal deployment problem as follows: given a set of available computing resources (VMs) from various Cloud Providers and an application to be deployed, find an assignment of components to VMs such that performance objectives are optimized (e.g. cost is minimized).

The problem is related to *bin-packing*, a fundamental problem in *combinatorial optimization*, which arises in many challenging problems from diverse application areas. Due to the importance of the bin-packing problem, there has been extensive research on developing mathematical formalisms, efficient algorithms, software systems, and applications (just to name a few: [8–12]). It can be formulated as follows [13]: given a set of bins $V_1, V_2, ..., V_M$ with the same size V and a list of N items with sizes $R_1, ..., R_N$ find: *(i)* the minimum number m of bins, and *(ii)* a m-partition $V_1 \cup ... \cup V_m$ of the set $\{1, ..., N\}$, such that the objects assigned to the bins do not exceed their capacity ($\sum_{i \in V_k} R_i \leq V, \ \forall k = \overline{1, m}$). The problem can be formulated as a constrained optimization problem (COP) as follows:

[1] https://github.com/dice-project.
[2] https://www.musa-project.eu.

$$\begin{array}{lll} \textit{Minimize } m = \sum_{k=1}^{M} v_k \textit{ subject to} & m \geq 1,\ a_{ik} \in \{0,1\},\ v_k \in \{0,1\},\ k = \overline{1,M} \\ & \sum_{i=1}^{N} R_i a_{ik} \leq V v_k, \\ & \sum_{k=1}^{M} a_{ik} = 1, \quad i = \overline{1,N} \\ \textit{where} & v_k = 1 \text{ if bin } k \text{ is used and } a_{ik} = 1 \text{ if item } i \text{ is placed in bin } k. \end{array}$$

However, in the context of component-based applications deployment in the Cloud:

1. bins (VMs) can have different capacity, which depends on the VMs offers;
2. the placement of items (components) in bins is limited not only by the capacity constraints, but also by the constraints induced by the components interactions;
3. the number of items is not known a priori (for component-based applications, several instances of a component can be deployed, depending on specific constraints on the number of instances);
4. the smallest cost (optimality criteria) is not necessarily obtained by minimizing the number of bins.

The observations above determine a new formulation of a COP which can be solved using search techniques. This, however, is feasible only if the size of the input (e.g. number of VM offers[3] and number of application components) is sufficiently small, since there exists an exponential number of combinations and, consequently, large instances are unfeasible within a reasonable time. For example, for our problem, a naive search technique based on backtracking is not scalable neither from the point of view of the application number of components nor from the point of view of the number of VMs offers, although one would expect scalability in the second case since the dimension of the search space depends only on the number of components and the offers are used only for the validation of the solutions satisfying the restrictions. The lack of scalability of the naive search techniques is the reason why these kind of problems were typically approached by using sub-optimal techniques, such as heuristics and meta-heuristics, as well as approximation algorithms (see [14] for a survey). On the other hand, optimal initial deployment would be desirable, thus scalable exact methods are of interest.

In this paper, we tackle the problem using exact approaches based on mathematical programming (MP) and OMT solving. Compared to naive search (e.g. backtracking), these use guided search techniques. The main difference between MP and OMT is the usage of different theoretical basis, namely algebra, respectively logical inferences for guiding the solution construction. Our goal is to study the scalability of the exact methods implemented in two state-of-the-art exact solvers CPLEX [15] (based on MP) and Z3 [16] (based on OMT solving), with emphasis on the OMT approach. The later mentioned approach scarcely appears in the context of COP related to Cloud [17] because this field of research has known tremendous advances only the last decade.

Our experimental results proved the lack of scalability of both types of solvers. However, by exploiting the symmetries of the underlying problem, we reduced the search space such that the scalability of both types of optimizers increased, most notable, by at least 2 orders of magnitude in the case of the OMT solver. This was achieved by:

1. systematically analyzing the symmetries which appear in the context of Cloud deployment applications;
2. design and integrate with state-of-the-art MP (CPLEX [15]) and OMT (Z3 [16]) tools static symmetry breakers for speeding-up the solution process.

The contributions of this paper are as follows:

1. we formalize the Cloud deployment problem (Section 3) by abstracting away the particularities of several realistic case studies (Section 2);
2. we propose a methodology analyzing the particularities of the problem with the aim of identifying search space reduction methods; these are methods exploiting the *symmetries* of the general Cloud deployment problem, respectively methods utilizing the graph representation of the interaction between the components of each application (Section 4);
3. we assess and compare the performance of two tools based on different theoretical background, namely mathematical programming (CPLEX [15]) and computational logic (Z3 [18]); we identified limits in their scalability and applied search space reduction methods aiming to improve their performance (Section 5).

Our paper shows the computational advantages of OMT methods especially when enhanced with symmetry breaking techniques, making them competitive with the long time used sub-optimal techniques.

The rest of the paper is organized as follows. In Section 2 we present real-world case studies which motivated the formalization of the deployment problem in the Cloud introduced in Section 3. In Section 4 we detail the methodology used

[3] The Crawler Engine that we implemented, gathered several hundreds of VMs offers from the main Cloud Providers.

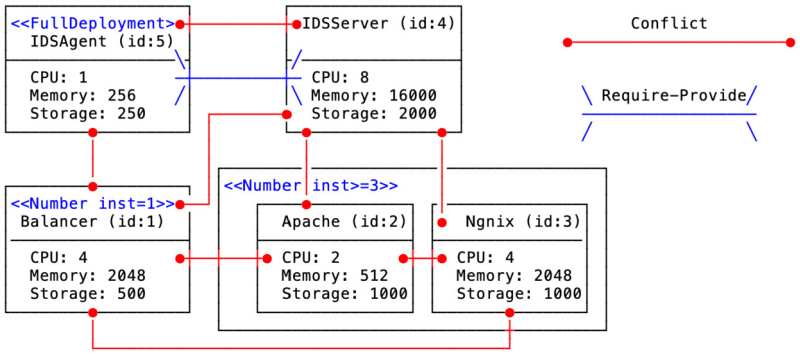


Fig. 1. Secure Web Container application.

for developing symmetry breaking methods whose efficiency is presented in Section 5. Finally, we present related work in Section 6 and draw conclusions and future research directions in Section 7.

2. Case studies

The case studies introduced in this section exemplify the following aspects: *(i)* different component characteristics and the rich interactions type in between; *(ii)* the kind of linear constraints used to express these interactions (see exemplifications in Section 3); *(iii)* the kind of solution we are searching for (see Section 4).

The case studies details were obtained after an in-depth architecture analysis. For example, during the *design phase*, besides specifying the components types, the developer is also asked to provide a high-level characterization of the components in terms of their resource demand (i.e. in terms of CPU cores, RAM and storage capacity). Each of the provided numbers are obtained after an analysis of the expected workload. For example, a value of 2 (cores) CPU means that the component needs this capacity in order to function at the right parameters. Afterwards, in the *functional design* phase, the developer has to identify the interactions/constraints between components.

2.1. Secure Web Container

The *Secure Web Container* [19] (Fig. 1) is a service which provides: *(i) resilience* to attacks and failures, by introducing redundancy and diversity techniques, and *(ii)* protection from unauthorized and potentially dangerous accesses, by integrating proper *intrusion detection* tools. Resilience can be implemented by a set of different Web Container components and a Balancer component, which is responsible for dispatching web requests to the active web containers to ensure load balancing. In the simplest scenario, there are two Web Containers (e.g. Apache Tomcat[4] and Nginx). Intrusion detection is ensured by the generation of intrusion detection reports with a certain frequency. It was implemented by deploying an IDSAgent, to be installed on the resources to be protected, and an IDSServer, which collects data gathered by the IDSAgents and performs the detection activities.
The constraints between application components are as follows.

- For Web resilience: *(i)* Any two of the Balancer, Apache and Nginx components cannot be deployed on the same machine (***Conflict*** constraint); *(ii)* Exactly one Balancer component has to be instantiated (***Deployment with bounded number of instances*** constraint, in particular ***equal bound***). *(iii)* The total number of instances for Apache and Nginx components must be at least 3 (level of redundancy) (***Deployment with bounded number of instances*** constraint, in particular ***lower bound***).
- For Web intrusion detection: *(i)* the IDSServer component needs exclusive use of machines (***Conflict*** constraint). *(ii)* There must be an IDSServer component additional instance every 10 IDSAgent component instances (***Require-Provide*** constraint). *(iii)* One instance of IDSAgent must be allocated on every acquired machine except where an IDSServer or a Balancer are deployed (***Full Deployment*** constraint).

We want to deploy this application in the Cloud with the minimal cost. There are multiple Cloud Providers that offer infrastructure services (virtual machines) in multiple heterogeneous configurations, including Amazon,[5] Google Cloud,[6] Microsoft Azure.[7] In fact, the Crawler Engine we implemented [20] gathered several hundreds of virtual machines offers

[4] http://tomcat.apache.org.
[5] https://aws.amazon.com.
[6] https://cloud.google.com/compute/.
[7] https://azure.microsoft.com/en-us/.

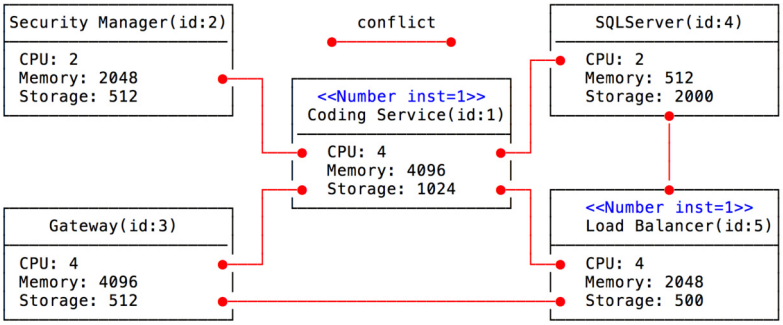


Fig. 2. Secure billing email service.

having different types, i.e. distinct hardware configurations (e.g. number of CPUs, memory, storage) and prices. Therefore the method used to solve the constraint optimization problem should be scalable with respect to the number of VM offers.

2.2. Secure billing email service

In the context of a web application ensuring a secure billing email service (Fig. 2) we consider an architecture consisting of 5 components: *(i)* a coding service (C_1), *(ii)* a software manager of the user rights and privileges (C_2), *(iii)* a gateway component (C_3), *(iv)* an SQL server (C_4) and *(v)* a load balancer (C_5). Component C_1 should use exclusively a virtual machine, thus it can be considered in ***conflict*** with all the other components. In such a case the original optimization problem can be decomposed in two subproblems, one corresponding to component C_1 and the other one corresponding to the other 4 components. The first problem is trivial: find the VM with the smallest price which satisfies the hardware requirements of component C_1.

The load balancing component should not be placed on the same machine as the gateway component and the SQL server (***Conflict*** constraint). On the other hand, only one instance of components C_1 and C_5 should be deployed while the other three components could have a larger number of instances placed on different virtual machines (***Deployment with bounded number of instances*** constraint, in particular ***equal bound***).

2.3. Wordpress

Wordpress open-source application is frequently used in creating websites, blogs and applications. We chose it in order to compare our approach to Zephyrus and Zephyrus2 deployment tools [21,17]. In [21,17], the authors present a high-load and fault tolerant Wordpress (Fig. 3) deployment scenario. The two characteristics are ensured by load balancing. One possibility is to balance load at the DNS level using servers like Bind[8]: multiple DNS requests to resolve the website name will result in different IPs from a given pool of machines, on each of which a separate Wordpress instance is running. Alternatively one can use as website entry point an HTTP reverse proxy capable of load balancing (and caching, for added benefit) such as Varnish. In both cases, Wordpress instances will need to be configured to connect to the same MySQL database, to avoid delivering inconsistent results to users. Also, having redundancy and balancing at the front-end level, one usually expects to have them also at the Database Management System (DBMS) level. One way to achieve that is to use a MySQL cluster, and configure the Wordpress instances with multiple entry points to it. In the deployment scenario considered by us, the following constraints must be fulfilled: *(i)* DNSLoadBalancer requires at least one instance of Wordpress and DNSLoadBalancer can serve at most 7 Wordpress instances (***Require-Provide*** constraint). *(ii)* HTTPLoadBalancer requires at least one Wordpress instance and HTTPLoadBalancer can serve at most 3 Wordpress instances (***Require-Provide*** constraint). *(iii)* Wordpress requires at least 2 instances of MySQL and MySQL can serve at most 3 Wordpress (***Require-Provide*** constraint). *(iv)* Only one type of Balancer must be deployed (***Exclusive deployment*** constraint). *(v)* Since Varnish exhibits load balancing features, it should not be deployed on the same VM as with another type of Balancer (***Conflict*** constraint). Moreover, Varnish and MySQL should not be deployed on the same VM because it is best practice to isolate the DBMS level of an application (***Conflict*** constraint). *(vi)* At least 2 instances of Varnish must be deployed too (***Deployment with bounded number of instances*** constraint, in particular ***lower bound***). *(vii)* At least 2 different entry points to the MySQL cluster (***Deployment with bounded number of instance*** constraint, in particular ***lower bound***). *(viii)* No more than 1 DNS server deployed in the administrative domain (***Deployment with bounded number of instances*** constraint, in particular ***upper bound***). *(ix)* Balancer components must be placed on a single VM, so they are considered to be in ***conflict*** with all the other components.

[8] https://www.isc.org/downloads/bind/.

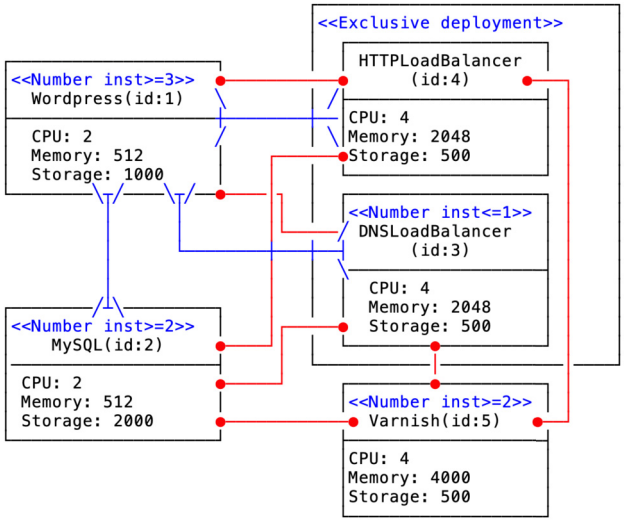


Fig. 3. Wordpress application.

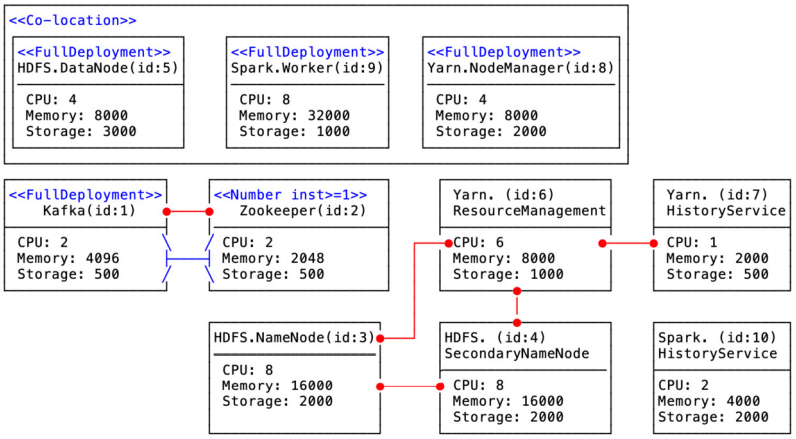


Fig. 4. Oryx2 application.

2.4. Oryx2

Oryx2 application (Fig. 4) is a realization of the lambda architecture, featuring speed, batch, and serving tiers, with a focus on applying machine learning models in data analysis, and deploys the latest technologies such as Apache Spark[9] and Apache Kafka.[10] It has a significant number of components interacting with each other and is highly used in practical applications. It consists of several components which can be distributed over thousands of VMs in the case of a full deployment. The main goal of Oryx2 is to take incoming data and use them to create and instantiate predictive models for various use-cases, e.g. movie recommendation. It is comprised of several technologies. Both the batch and serving layer are based on Apache Spark which in turn uses both Apache Yarn[11] for scheduling and Apache HDFS as a distributed file system. For a processing pipeline Oryx2 uses Apache Kafka with at least two topics; one for incoming data and one for model update. Apache Zookeeper[12] is used by Kafka for broker coordination. All of the aforementioned technologies have subservices with a minimum system requirement and recommended deployment as of Fig. 4. The constraints corresponding to the interactions between the components are described in the following. *(i)* Components HDFS.DataNode and Spark.Worker must the deployed on the same VM (***Co-location***). In this scenario, we also collocated Yarn.NodeManager because we used Yarn as a scheduler for Spark jobs. *(ii)* Components Kafka and Zookeeper, HDFS.NameNode and HDFS.SecondaryNameNode, YARN.Re-

[9] https://spark.apache.org/.
[10] https://kafka.apache.org/.
[11] http://hadoop.apache.org/.
[12] https://zookeeper.apache.org/.

sourceManagement and HDFS.NameNode, HDFS.SecondaryNameNode, YARN.HistoryService are, respectively, in ***conflict***, that is, they must not be placed on the same VM. *(iii)* Components HDFS.DataNode, YARN.NodeManager and Spark.Worker must be deployed on all VMs except those hosting conflicting components (***Full Deployment***). *(iv)* In our deployment, we consider that for one instance of Kafka there must be deployed exactly 2 instances of Zookeeper (***Require-Provide*** constraint). There can be situations, however, when more Zookeeper instances are deployed for higher resilience. *(v)* A single instance of YARN.HistoryService, respectively Spark.HistoryService should be deployed (***Deployment with bounded number of instances*** constraint, in particular ***equal bound***).

3. The problem

To describe the problem in a formal way, we consider a set of N interacting components, $C = \{C_1, \ldots, C_N\}$, to be assigned to a set of M virtual machines, $V = \{V_1, \ldots, V_M\}$. Each component C_i is characterized by a set of requirements concerning the hardware resources. Each virtual machine, V_k, is characterized by a *type*, which is comprised by hardware/software characteristics and leasing price. There are also *structural constraints* describing the interactions between components (see Section 3.2). The problem is to find:

1. assignment matrix a with binary entries $a_{ik} \in \{0, 1\}$ for $i = \overline{1, N}$, $k = \overline{1, M}$, which are interpreted as follows:

$$a_{ik} = \begin{cases} 1 & \text{if } C_i \text{ is assigned to } V_k \\ 0 & \text{if } C_i \text{ is not assigned to } V_k. \end{cases}$$

2. the type selection vector t with integer entries t_k for $k = \overline{1, M}$, representing the type (from a predefined set) of each VM leased.

such that: *(i)* the structural constraints, *(ii)* the hardware requirements (capacity constraints) of all components are satisfied and *(iii)* the purchasing/ leasing price is minimized.

For instance, in the case of a Secure Web Container Service (Section 2.1), the problem corresponding to 5 components and to a prior estimation of the number of VMs equal to 6, a solution can be (1), respectively (2).

$$a = \begin{pmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \end{pmatrix} \tag{1}$$

$$t = [12, 12, 13, 13, 15, 0]. \tag{2}$$

The *structural constraints* are *application-specific* (see Section 3.2) and derived in accordance with the analysis of the case studies from Section 2. *General constraints* (see Section 3.1) are always considered in the formalization and are related to the: *(i) basic allocation* rules, *(ii) occupancy* criteria, *(iii)* hardware *capacity* of the VM offers, *(iv) link* between the VM offers and the components hardware/software requirements.

The problem to solve can be stated as a COP as follows[13]:

Minimize	$\sum_{k=1}^{M} v_k \cdot p_k$	
Subject to	$a_{ik} \in \{0, 1\}$, $v_k \in \{0, 1\}$	
	$t_k \in$ predefined finite set of natural numbers	$\forall i = \overline{1, N}, \forall k = \overline{1, M}$
General	***constraints***	
Basic allocation	$\sum_{k=1}^{M} a_{ik} \geq 1$	$\forall i = \overline{1, N}$
Occupancy	$\sum_{i=1}^{N} a_{ik} \geq 1 \Rightarrow v_k = 1$	$\forall k = \overline{1, M}$
Capacity	$\sum_{i=1}^{N} a_{ik} \cdot R_i^h \leq F_{t_k}^h$	$\forall k = \overline{1, M}, \forall h = \overline{1, H}$
Link	$v_k{=}1 \wedge t_k{=}o \Rightarrow \bigwedge_{h=1}^{H} \left(r_k^h{=}F_{t_k}^h\right) \wedge p_k{=}P_{t_k}$	$\forall o = \overline{1, O},\ O \in \mathbb{N}^*$
	$\sum_{i=1}^{N} a_{ik} = 0 \Rightarrow t_k = 0$	$\forall k = \overline{1, M}$

[13] As of the definition of a VM type, t_k includes p_k. If $t_k = 0$ then the machine k is not occupied.

Application-	***specific constraints***	
Conflicts	$a_{ik} + a_{jk} \leq 1$	$\forall k = \overline{1, M}, \forall (i, j)\ \mathcal{R}_{ij} = 1$
Co-location	$a_{ik} = a_{jk}$	$\forall k = \overline{1, M}, \forall (i, j)\ \mathcal{D}_{ij} = 1$
Exclusive	*deployment*	
	$\mathcal{H}\left(\sum_{k=1}^{M} a_{i_1 k}\right) + ... + \mathcal{H}\left(\sum_{k=1}^{M} a_{i_q k}\right) = 1$	for fixed $q \in \{1, ..., N\}$
		$\mathcal{H}(u) = \begin{cases} 1 & u > 0 \\ 0 & u = 0 \end{cases}$
Require-	*Provide*	
	$n_{ij} \sum_{k=1}^{M} a_{ik} \leq m_{ij} \sum_{k=1}^{M} a_{jk}$	$\forall (i, j) \mathcal{Q}_{ij}(n_{ij}, m_{ij}) = 1$
	$0 \leq n \sum_{k=1}^{M} a_{jk} - \sum_{k=1}^{M} a_{ik} < n$	$n, n_{ij}, m_{ij} \in \mathbb{N}^*$
Full deployment	$\sum_{k=1}^{M} \left(a_{ik} + \mathcal{H}\left(\sum_{j, \mathcal{R}_{ij}=1} a_{jk} \right) \right) = \sum_{k=1}^{M} v_k$	
Deployment with	*bounded number of instances*	
	$\sum_{i \in \overline{C}} \sum_{k=1}^{M} a_{ik} \langle \text{op} \rangle n$	$\lvert\overline{C}\rvert \leq N, \langle \text{op} \rangle \in \{=, \leq, \geq\}, n \in \mathbb{N}$

where:

- $\mathcal{R}_{ij} = 1$ if components i and j are in conflict (can not be placed in the same VM);
- $\mathcal{D}_{ij} = 1$ if components i and j must be co-located (must be placed in the same VM);
- $\mathcal{Q}_{ij}(n, m)$=1 if C_i requires at least n instances of C_j and C_j can serve at most m instances of C_i;
- $R_i^h \in \mathbb{N}^*$ is the hardware requirement of type h of the component i;
- $F_{t_k}^h \in \mathbb{N}^*$ is the hardware characteristic h of the VM of type t_k.

3.1. General constraints

The *basic allocation rules* specify that each component C_i must be allocated to at least one VM, except those being in *Exclusive Deployment* relation (see below).

$$\sum_{k=1}^{M} a_{ik} \geq 1 \qquad \forall i = \overline{1, N}$$

Capacity constraints specify that the total amount of a certain resource type h required by the components hosted on a particular VM does not overpass the corresponding resource type of a VM offer.

$$\sum_{i=1}^{N} a_{ik} \cdot R_i^h \leq F_{t_k}^h \qquad \forall k = \overline{1, M},\ \forall h = \overline{1, H},\ \forall t_k = \overline{1, O},\ O \in \mathbb{N}^* \tag{3}$$

For example, the VM offer of type t_1=1, characterized by CPU, memory, storage and price is encoded as $(F_1^1, F_1^2, F_1^3, P_1) =$ (64, 976 GB, 1 GB, 8.403 USD/h).

In order to have a sound formalization, one also needs to link a type of a VM offer to each of the occupied VMs. Each t_k refers to a tuple $(r_k^1, r_k^2, ..., r_k^H, p_k)$:

$$v_k{=}1 \wedge t_k{=}o \Rightarrow \bigwedge_{h=1}^{H} \left(r_k^h{=}F_{t_k}^h \right) \wedge p_k{=}P_{t_k} \qquad \forall k = \overline{1, M},\ \forall o = \overline{1, O},\ O \in \mathbb{N}^* \tag{4}$$

In our approach M denotes an upper estimation (see Section 4.1) for the number of VMs needed for deployment. Such estimation might actually be higher than the optimum. Hence, the following constraint is needed:

$$\sum_{i=1}^{N} a_{ik} = 0 \Rightarrow t_k = 0, \quad \forall k = \overline{1, M}$$

Vector v is the binary occupancy vector defined as:

$$\sum_{i=1}^{N} a_{ik} \geq 1 \Rightarrow v_k = 1, \qquad \forall k = \overline{1, M} \tag{5}$$

i.e. v_k is 1 if machine V_k is used and is 0 otherwise.

3.2. Application-specific constraints

We identified two main types of application-specific constraints regarding the components: those concerning the *interactions* between components (conflict, co-location, exclusive deployment) and those concerning the *number of instances* (require-provide, full deployment, deployment with a bounded number of instances).

Conflict. This case corresponds to situations when there are conflictual components which cannot be deployed on the same VM. Considering that all conflicts between components are encoded in a matrix $\mathcal{R}$ (i.e. $\mathcal{R}_{ij} = 1$ if C_i and C_j are conflictual components and $\mathcal{R}_{ij} = 0$ otherwise), the constraints can be described as a set of linear inequalities:

$$a_{ik} + a_{jk} \leq 1, \qquad k = \overline{1, M}, \text{ for all } (i, j) \text{ such that } \mathcal{R}_{ij} = 1.$$

It should be noted that this type of constraints usually induces an increase in the number of VMs.
For example, for the Wordpress application, Varnish component exhibits load balancing features. Hence, it should not be deployed on the same VM with HTTPLoadBalancer or DNSLoadBalancer. Moreover, Varnish and MySQL should not be deployed on the same VM because it is best practice to isolate the DBMS level of an application. Therefore, based on the notations in Fig. 3, where each component has an assigned identifier, the corresponding constraints are:

$$a_{5k} + a_{ik} \leq 1, \quad i \in \{2, 3, 4\}, \quad k = \overline{1, M}$$

Co-location. This means that the components in the collocation relation should be deployed on the same VM. The co-location relation can be stored in a matrix $\mathcal{D}$ (i.e. $\mathcal{D}_{ij} = 1$ if C_i and C_j should be collocated and $\mathcal{D}_{ij} = 0$ otherwise) and the constraints can be described as a set of equalities:

$$a_{ik} = a_{jk}, \qquad k = \overline{1, M}, \text{ for all } (i, j) \text{ such that } \mathcal{D}_{ij} = 1$$

The number of VMs needed decreases with the increase of the number of co-located components.
For example, for the Oryx2 application, components HDFS.DataNode and Spark.Worker must be deployed on the same VM. In this scenario, we also co-located Yarn.NodeManager because we used Yarn as a scheduler for Spark jobs:

$$a_{ik} = a_{9k}, \quad i \in \{5, 8\}, \quad k = \overline{1, M}$$

Exclusive deployment. There are cases when from a set of q components $\{C_{i_1}, C_{i_2}, ..., C_{i_q}\}$ only one should be deployed in a deployment plan. Such a constraint can be described as:

$$\mathcal{H}\left(\sum_{k=1}^{M} a_{i_1 k}\right) + \mathcal{H}\left(\sum_{k=1}^{M} a_{i_2 k}\right) + ... + \mathcal{H}\left(\sum_{k=1}^{M} a_{i_q k}\right) = 1,$$

where $\mathcal{H}$ is a function defined as: $\mathcal{H}(u) = 1$ if $u > 0$ and $\mathcal{H}(u) = 0$ if $u = 0$.
For example, for the Wordpress application, only one type of Balancer must be deployed (the Balancer components are HTTPLoadBalancer and DNSLoadBalancer). If HTTPLoadBalancer is deployed, a caching component, in our case Varnish, should also be deployed leading to a different set of conflicts:

$$\mathcal{H}\left(\sum_{k=1}^{M} a_{3k}\right) + \mathcal{H}\left(\sum_{k=1}^{M} a_{4k}\right) = 1 \quad \text{and} \quad \mathcal{H}\left(\sum_{k=1}^{M} a_{3k}\right) + \mathcal{H}\left(\sum_{k=1}^{M} a_{5k}\right) = 1$$

Require-Provide. A special case of interaction between components is when one component requires some functionalities offered by other components. Such an interaction induces constraints on the number of instances corresponding to the interacting components as follows: *(i)* C_i requires (consumes) at least n_{ij} instances of C_j and *(ii)* C_j can serve (provides) at most m_{ij} instances of C_i. This can be written as:

$$n_{ij} \sum_{k=1}^{M} a_{ik} \leq m_{ij} \sum_{k=1}^{M} a_{jk}, \; n_{ij}, m_{ij} \in \mathbb{N}. \tag{6}$$

For example, for Wordpress application, the Wordpress component (C_1) requires at least three instances of MySQL and MySQL (C_2) can serve at most 2 Wordpress instances, leading to the constraint:

$$3\sum_{k=1}^{M} a_{1k} \leq 2\sum_{k=1}^{M} a_{2k}, \quad k = \overline{1, M}.$$

A related case is when for each set of n instances of component C_i a new instance of C_j should be deployed. This can be described as:

$$0 \leq n\sum_{k=1}^{M} a_{jk} - \sum_{k=1}^{M} a_{ik} < n, \ n \in \mathbb{N} \tag{7}$$

This constraint cannot be deduced from (6) because of the following. Taking in (6) $n_{ij} = 1$, we obtain an expression meaning that for m_{ij} instances of C_j one should have at least one instance of C_i (but there can be more). (7) is more specific requiring exactly one instance of C_j.

Full deployment. There can be also cases when a component C_i must be deployed on all leased VMs (except on those which would induce conflicts on components). This can be expressed as:

$$\sum_{k=1}^{M} \left(a_{ik} + \mathcal{H}\left(\sum_{j, \mathcal{R}_{ij}=1} a_{jk} \right) \right) = \sum_{k=1}^{M} v_k$$

where $\mathcal{R}$ is the conflicts matrix and $\mathcal{H}$ is defined as above.
For example, for the Oryx2 application, components HDFS.DataNode (C_5), YARN.NodeManager (C_8) and Spark.Worker (C_9) must be deployed on all VMs except those hosting conflicting components. Since $\mathcal{H}\left(\sum_{j, \mathcal{R}_{ij}=1} a_{jk} \right) = 0$, we have $\sum_{k=1}^{M} a_{ik} = \sum_{k=1}^{M} v_k$, for $i \in \{5, 8, 9\}$.
Note that we do not allow, in the application description, the full deployment of two conflicting components.

Deployment with bounded number of instances. There are situations when the number of instances corresponding to a set, $\overline{C}$, of deployed components should be equal, greater or less than some values. These types of constraints can be described as follows:

$$\sum_{i \in \overline{C}} \sum_{k=1}^{M} a_{ik} \ \langle \text{op} \rangle \ n, \quad \langle \text{op} \rangle \in \{=, \leq, \geq\}, \ n \in \mathbb{N}$$

For example, for the Secure Web Container application, the total amount of instances of components Apache (C_2) and Nginx (C_3) must be at least 3 (level of redundancy):

$$\sum_{k=1}^{M} a_{2k} + \sum_{k=1}^{M} a_{3k} \geq 3$$

4. Solving approach

Our solving approach is based on the following methodology: *(i)* Estimate an upper bound on the number of VMs needed for the application deployment (Section 4.1). *(ii)* Analyze the application-specific constraints, in particular co-location and conflicts, and adapt the formalization in such a way that constraints are implicitly satisfied (co-location) or the search space is reduced (conflicts) (Section 4.2). *(iii)* Analyze the symmetries of the problem and identify symmetry breaking strategies (Section 4.3). *(iv)* Select the optimization method (Section 4.4).

4.1. Prior estimation of the number of virtual machines

The number of decision variables (elements of the assignment matrix a and of the type selection vector t) depends on the number of VMs, M, taken into account in the deployment process. However, the optimal number of used VMs is also unknown, thus a prior estimation of a (tight) upper bound is required. In the case of the traditional bin-packing problem, such an upper bound is given by the number of items. In the case of resource allocation class of problems addressed in this paper, since the number of instances per component is also unknown, estimating an upper bound for the number of VMs is not trivial. In order to estimate M we solve a *surrogate* optimization problem which takes into account only constraints

involving number of instances (i.e. *Require-Provide* and *Deployment with bounded number of instances*) and minimizes the total number of instances:

$$\begin{aligned}
&\textbf{\textit{Minimize}} && M = \sum_{i=1}^{N} \nu_i \\
&\textbf{\textit{Subject to}} && \nu_i \in \mathbb{N}^*, && i = \overline{1, N} \\
&\textit{Require-Provide} && n_{ij}\nu_i \le m_{ij}\nu_j && \forall (i, j)\, \mathcal{Q}_{ij}(n_{ij}, m_{ij}) = 1 \\
& && 0 \le n\nu_j - \nu_i < n \\
&\textit{Bounded number of instances} && \sum_{i \in \overline{C}} \nu_i \ge n && |\overline{C}| \le N
\end{aligned}$$

As M is set to the sum of the number of instances estimated by solving the above surrogate problem, it means that it corresponds to the case when each instance will be assigned to a distinct machine which would correspond to the case when all components would be in conflict. Since in real world cases usually not all components are in conflict, it follows that M is an upper bound estimate of the number of virtual machines required to satisfy all application-specific constraints.

4.2. Analysis of the application-specific constraints

Some constraints can be satisfied by a proper encoding of the decision variables (as that referring to the fact that only one instance of a component is deployed on a VM) while other ones can be exploited in order to reduce the search space by redefining the models.

4.2.1. Co-location-type constraints

For instance, the co-location-type constraints can be exploited by combining all co-located components in a hyper-component and redefining the existing constraints. Let us consider that $\overline{C} = \{C_{i_1}, C_{i_2}, \ldots, C_{i_L}\}$ is a set of components which should be co-located, i.e. for each instance of one of the components deployed on a VM, an instance of all the other components should be deployed on the same machine. Thus all components in $\overline{C}$ will have the same number of deployed instances and the original problem of assigning N components to VMs can be reformulated as the problem of assigning $N - L + 1$ components (as L components will be replaced with one hyper-component, C_{i^*}). The original constraints involving elements of $\overline{C}$ will be also reorganized as follows:

Conflicts. All conflict type constraints involving elements C_{i_l} of $\overline{C}$ will be replaced with one constraint involving C_{i^*}, i.e. if $R_{i_l j} = 1$ then $R_{i^* j} = 1$.

Exclusive Deployment. If an element C_{i_l} of the hyper-component C_{i^*} is in an exclusive deployment relation with a component C_j then C_j will be excluded, i.e. C_{i^*} is preferred.

Full Deployment. If at least one component of the hyper-component C_{i^*} appears in full deployment constraint then such a constraint should be added for C_{i^*}.

Require-provide. In all require-provide constraints involving components from $\overline{C}$ these components will be replaced with the hyper-component C_{i*}.

Capacity related constraints. The capacity constraints (e.g. CPUs, memory size, storage) concerning the components from $\overline{C}$ will be aggregated by sum in unique constraints involving the hyper-component.

4.2.2. Conflict-type constraints

When there are conflictual components, the conflict graph can be used to identify components which should be placed on different machines. This type of constraints can be further exploited by fixing the values of the decision variables which correspond to some of the conflictual components. More specifically, in a first step, all cliques which exist in the conflict graph are identified, i.e. subsets of components which are fully conflicting, meaning that their instances should be deployed on different VMs. Then the clique $\overline{G} = \{C_{j_1}, C_{j_2}, \ldots, C_{j_L}\}$ is selected, that is the clique with the largest deployment size, i.e. the largest number of instances ($\nu_{j_1} + \nu_{j_2} + \ldots + \nu_{j_L}$). Since the instances of all components belonging to $\overline{G}$ should be assigned to different machines, one can fix values of the assignment variables as follows. Each of the ν_{j_q} instances of component C_{j_q} in $\overline{G}$ is assigned sequentially to the first available machine, i.e. the following constraints are explicitly set:

$$a_{j_q (l + Q_{q-1})} = 1, \qquad \text{for } l = \overline{1, \nu_{j_q}} \tag{8}$$

$$a_{j_q l} = 0, \qquad \text{for } l = \overline{1, Q_{q-1}} \text{ and } l = \overline{Q_q + 1, Q_L}, \tag{9}$$

where the value $Q_q = \nu_{j_1} + \nu_{j_2} + \ldots + \nu_{j_q}$ denotes the number of machines already occupied by instances corresponding to components $\{C_{j_1}, C_{j_2}, \ldots, C_{j_q}\}$. Since, the values ν_{j_q} obtained by solving the surrogate problem described in subsection 4.1 might represent over-estimations (e.g. particularly for the components involved in constraints related to bounded number of instances) a conservative approach would be to fix variables corresponding to just one instance for each of the components belonging to the selected clique $\overline{G}$.

Based on the observations above, as well as on the analysis of our case studies, we grouped the decision variables into three main categories: *(i)* variables with fixed values (as those set above); *(ii)* variables bounded by constraints (the variables

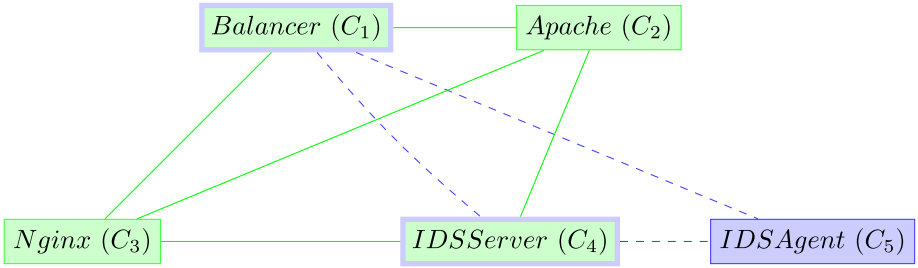


Fig. 5. Secure Web Container conflict graph. The components with green background belong to the clique $\overline{G}$. (For interpretation of the colors in the figure, the reader is referred to the web version of this article.)

Table 1
Fixing variables in the assignment matrix a of the Secure Web Container use case.

	VM_1	VM_2	VM_3	VM_4	VM_5	VM_6
C_1	[1]	[0]	[0]	[0]	0	0
C_2	[0]	[0]	[1]	[0]	1	0
C_3	[0]	[0]	[0]	[1]	0	0
C_4	[0]	[1]	[0]	[0]	0	0
C_5	[0]	[0]	1	1	1	0

for which their values are determined by solving some of these constraints); *(iii)* variables free of constraints, for which the values are mainly controlled through the optimization criterion (e.g. components which fit into any offered VM and are not involved in constraints describing the interaction between components).

Let us consider the Secure Web Container use case (see Fig. 1). The surrogate optimization problem presented in Section 4.1 estimates a maximum of 6 VMs for solving the problem. The conflict graph corresponding to the application described in Fig. 1) contains two cliques: [Balancer, Apache, Nginx, IDSServer] and [Balancer, IDSServer, IDSAgent], the first one having the largest deployment size hence playing the role of $\overline{G}$ (see Fig. 5). Based on the variable fixing rules described above, most of the decision variables can be fixed, as is illustrated in Table 1, where framed elements correspond to fixed values, and the other elements correspond to variables which are bounded by constraints (e.g. Full deployment in the case of $IDSAgent$, Deployment with bounded number of instances for the set $\{Apache, Nginx\}$). Note that the precise number of instances of *Apache* and *Nginx* is not known at the moment of variables fixing; we only know that the sum of their instances should be at least 3. Hence only one instance for *Apache*, respectively, *Nginx* is fixed. In this example, we also explored the fact that C_1 is in conflict with C_5, although C_5 is not part of $\overline{G}$. In this example there are no free of constraints variables, but it would be possible to take into account a component which is not involved in structural constraints (e.g. conflicts, co-location, exclusive or full deployment, require-provide) but can be beneficial from a functional point of view (e.g. a monitoring component). In such a case, different deployment plans having the same cost, but different assignments for the free of constraints component, can be generated.

4.3. Symmetries and symmetry breakers

Realistic Cloud applications might involve the deployment of a large number of components instances on VMs selected from a large pool of offers. This leads to the necessity of solving optimization problems with a large search space, which, however might contain equivalent solutions. The search space can be limited by reducing the number of decision variables (as is illustrated in Section 4.2.1), by reducing the number of unassigned variables (as is illustrated in Section 4.2.2) or by breaking the symmetries related to the decision variables as discussed in this section.

In the following, symmetries which appear in Cloud deployment problems are described and appropriate symmetry breaking strategies are identified. For a self-contained presentation, we first introduce some theoretical notions.

4.3.1. Preliminaries

A *matrix model* [22] is a constraint program that contains one or more matrices of decision variables. *Symmetries* in constraint satisfaction/optimization problems, in general, and in matrix models, in particular, are a key problem since search can revisit equivalent states many times. In order to deal with symmetries, one must first define what they are. We use the definition from [23], namely *a symmetry is a bijection on decision variables that preserves solutions and non-solutions*. Two variables are *indistinguishable* if some symmetry interchanges their roles in all solutions and non-solutions. These are *variable symmetries.* The definition can be extended also to *value symmetries*, i.e. symmetries that permute only the values of variables.

For matrix models, symmetry often occurs because groups of objects within a matrix are *indistinguishable*. This leads to *row/column symmetries*. Two rows/columns are indistinguishable if their variables are pairwise identical due to a row/column symmetry. A matrix model has *row/column symmetry* iff all the rows/columns of one of its matrices are *indistinguishable*. A matrix model has *partial row/column symmetry* iff strict subset(s) of the rows/columns of one of its matrices are *indistinguishable*. Partial row/column symmetry are more often encountered in Cloud deployment problems, as explained in this section.

Elimination of equivalent states, problem known as *symmetry breaking*, has, most of the times, a positive impact in the computation time of the problem solution process. Symmetries can be eliminated by using *symmetry breaking* techniques which can be categorized as follows [24, Chapter 10]:

1. *Reformulation* means that the problem is remodeled to eliminate some or all symmetries. It proved to be very efficient method for breaking symmetry, but unfortunately there is no known systematic procedure for performing the remodeling process in general.
2. *Static symmetry breaking* adds the so-called *symmetry breaking constraints* before search starts, hence making some symmetric solutions unacceptable while leaving at least one solution in each symmetric equivalence class.
3. *Dynamic symmetry breaking* removes symmetries dynamically during search, adapting the search procedure appropriately.

In this paper we use *static symmetry breaking* as detailed in Section 5.

A natural way to break symmetry is to order the symmetric objects. To break row/column symmetry, one can simply order the rows/columns lexicographically. We say that the *rows/columns in a matrix are lexicographically ordered if each row/column is lexicographically smaller than the previous*.

Symmetries of Cloud deployment problems can be eliminated in a similar manner, as explained in the following.

4.3.2. Column symmetries

The *column symmetries* for the Cloud deployment problem from Section 3 are determined by the decision variables a and t. In defining the column symmetries one might exploit the different point of views of the problem. On one hand, the variables t_k, $\forall k = \overline{1, M}$, representing the type of leased VMs, might be considered indistinguishable, thus exhibit symmetry. At the same time but independently, the columns of the assignment matrix a, representing VMs, might be considered indistinguishable. These correspond to *full variable symmetry*. Symmetry breakers are based on the idea of ordering the columns, for example:

(i) decreasing by the number of components: $\sum_{i=1}^{N} a_{ik} \geq \sum_{i=1}^{N} a_{i(k+1)}, \quad \forall k = \overline{1, N-1}$

(ii) decreasing by lexicographic order of columns: $a_{\star k} \succ_{lex} a_{\star(k+1)}$ where $a_{\star k}$ denotes the column k.

(iii) decreasing by lexicographic order of tuples containing the price and the hardware characteristics of a VM, e.g. CPU number, memory, storage.

On the other hand, after t_k, $\forall k = \overline{1, M}$ are assigned values, only the VMs of the same type are indistinguishable. This means that only the columns of the assignment matrix a corresponding to VMs of the same type are indistinguishable. This is *partial symmetry* determined by the fact that we turn *partial value symmetry* for t into *partial variable symmetry for a.* Symmetry breakers are also based on the idea of ordering the columns but with some restrictions, for example:

(i) decreasing by the number of components for columns representing VMs of the same type:

$$(t_k = t_{k+1}) \Rightarrow \left(\sum_{i=1}^{N} a_{ik} \geq \sum_{i=1}^{N} a_{i(k+1)} \right), \quad \forall k = \overline{1, N-1} \tag{10}$$

(ii) decreasing by lexicographic order of columns for columns representing VMs of the same type

$$(t_k = t_{k+1}) \Rightarrow \left(a_{\star k} \succ_{lex} a_{\star(k+1)} \right), \text{ where } a_{\star k} \text{ denotes the column } k. \tag{11}$$

4.3.3. Row symmetries

The *row symmetries* for the Cloud deployment problem from Section 3 are determined by different viewpoints on the assignment matrix a. On one hand, the problem has row symmetry if by applying a permutation on the row labels of a (i.e. on the set of components $C_1, C_2, \ldots, C_N$), the assignment matrix corresponds to an equivalent solution (all constraints are satisfied and the value of the optimization criteria is the same).

The main application-specific constraints which induce row symmetry are that related to *conflicts*, as any two components being in a *conflict* relationship are indistinguishable. One can break this kind of symmetry by computing the clique with maximal deployment size in which all components are pairwise conflictual hence can not be placed on the same VM. More details were given in Section 4.2.2.

On the other hand, in the case when all components of an application are identical from the point of view of the hardware requirements and there are no application-specific constraints, the problem has full symmetry with respect to the rows. This means that any of the $N!$ permutations of the assignment matrix rows will correspond to an equivalent solution. However, it is rarely this case of Cloud deployment applications. Therefore we are dealing rather with partial row symmetry instead of full row symmetry.

4.4. Optimization approaches

Optimization problems originated from resource management problems can be tackled by *exact* (*constraints programming*, *mathematical programming*) or *inexact* (*meta-heuristic algorithms*) methods. Inexact methods are highly used in the literature because of their low computational time. However, these methods are suboptimal and there are no theoretical results which allow to estimate how far from the real optimum the solution is. The benefit of exact techniques is that they guarantee the optimal solution, with the disadvantage of higher computational time.

In this paper we used mathematical programming (MP) and constraint programming, in particular OMT solving. The main difference between these two approaches is the usage of different theoretical basis, namely algebra, respectively logical inferences for solution construction.

4.4.1. Mathematical programming

Mathematical programming is a branch in the field of operations research dealing with groups of methods for various optimization problems, particularly linear and quadratic optimization problems. The problem addressed in this paper belongs to the class of integer linear programming problems for which there are well established solving methods.

In our experiments, we used CPLEX solver [15]. It is the first commercial linear optimizer on the market to be written in the C programming language started being developed 20 years ago. It implements efficient algorithms solving integer programming, mixed integer programming, and quadratic programming problems. Distinct features of CPLEX, which to the best of our knowledge are not available for OMT solvers, are: *(i)* it allows to specify bounds on variables at the moment of their declaration; this avoids adding additional logical constraints to the model defining these bounds, *(ii)* it exhibits performance variability which we exploited by ordering the constraints and by using pre-processing parameters (see Section 5.1.2). A drawback which we encountered when using CPLEX to solve the problem was an explosion of additional variables dynamically generated by the solver, which influences negatively the processing time. This happens because CPLEX introduces a new variable for each sum appearing in constraints (e.g. sum of decision variables corresponding to the same column as in (6)), even if the same sum appears in several constraints, hence the same variable would have been used. In order to avoid such a variables explosion, the set of occupancy decision variables (i.e. vector v in the problem description) has been explicitly introduced in the CPLEX problem specification.

4.4.2. Satisfiability and optimization modulo theories

SMT solving is an extension of satisfiability (SAT) solving by introducing the possibility of stating constraints in some expressive theories, for example arithmetic, data structures, bit-vector expressions and their valid combinations. The idea behind SMT solving is that, given a formula in a certain theory, this is translated into a propositional formula. This is checked for satisfiability using a SAT solver. If unsatisfiability can not be deduced, then a candidate model (variables assignment) is fed to the theory solver. If this candidate model makes the initial formula true, SAT is returned meaning that the initial formula is true for the respective model. If SAT can not be deduced then the theory solver backtracks trying another candidate model. This process is repeated until no more candidate models are found.

For our problem, we needed a SMT solver which exhibits optimization features, the so-called *Optimization Modulo Theories (OMT) solvers*. There are not too many options in this regard: *(i)* OptiMathSAT [25] uses an inline architecture in which the SMT solver (MathSAT5[14]) is run only once and its internal SAT solver is modified to handle the search for the optima, *(ii)* Symba [26] and νZ [16] both are based on an offline architecture in which the SMT solver Z3 [18] is incrementally called multiple times as a black-box. Since Symba is not actively maintained and, in our previous work νZ outperformed OptiMathSAT [27], in this paper we will use νZ. We refer to it as $Z3$ being an integrating part of it. It is built on top of the mature SMT solver Z3, it was widely used in various application domains and offers a Python API. It takes as an input SMT formulas (constraints and objectives) which are simplified to Pseudo-Boolean Optimization (PBO) constraints. The PBO solver implements a wide range of methods for simplification and generating conflicting clauses, and compiling them into small sorting circuits. At the heart of the tool lies the dual simplex algorithm which is also used to prune branches.

OMT is similar to constraint programming methods as they both use logical inferences to construct de solution. Hence, encoding the problem in OMT formalism meant almost a one to one translation of the constraints introduced in Section 3.

5. Experimental analysis

The goal of the experimental analysis is two fold. On one hand, we want to asses the *scalability* of state-of-the-art general MP and OMT tools, namely CPLEX [15], respectively Z3 [16], in solving COPs corresponding to the case studies from

[14] http://mathsat.fbk.eu.

Section 2. Tests (see Section 5.2) revealed that the naive application of general MP and OMT techniques is not sufficient to solve realistic Cloud deployment applications. Hence, on the other hand, we evaluate the *effectiveness* of various static symmetry breaking techniques in improving the computational time of solving these problems (see Section 5.1). The scalability and effectiveness are evaluated from two perspectives: number of VMs offers, respectively number of deployed instances of components. For *Secure Web Container*, *Secure Billing Email* and *Oryx2* applications, we considered up to 500 VMs offers. Additionally, for the *Wordpress* application, we considered up to 12 instances of the Wordpress component to be deployed. The set of offers was crawled from the Amazon CPs offers list.

5.1. Experimental settings

In this section we present and motivate the plethora of symmetry breakers tested on two types of optimization tools for optimization, as well as the characteristics of the hardware we ran the experiments on.

5.1.1. Selected symmetry breaking strategies

Aiming to reduce the search space size, a set of strategies have been selected in order to exploit the particularities of the problem: *(i)* the VMs needed for application deployment might have different characteristics; *(ii)* applications components might be in conflict hence conflict-type constraints can be exploited; *(iii)* the number of instances to be deployed is unknown.

Our approach is incremental and experimental: we start with traditional symmetry breakers that have been used for other problems related to bin-packing and combine them with the aim of further search space reduction.

Price-based ordering (PR). This strategy aims to break symmetry by ordering the vector containing the types of used VMs decreasingly by price, i.e. $p_k \geq p_{k+1},\ k = \overline{1, M-1}$. This means that the solution will be characterized by the fact that the columns of the assignment matrix will be ordered decreasingly by the price of the corresponding VMs.

Lexicographic ordering (LX). This corresponds to the traditional strategy aiming to break column-wise symmetries. The constraints to be added aiming to ensure that two columns, k and $(k+1)$ are in a decreasing lexicographic order, i.e. $a_{*k} \succ_{lex} a_{*(k+1)}$, are:

$$\bigwedge_{l=1}^{i-1}(a_{lk} = a_{l(k+1)}) \Longrightarrow a_{ik} \geq a_{i(k+1)}, \quad \forall i = \overline{1, N} \tag{12}$$

Price-based and lexicographic ordering (PRLX). The columns corresponding to VMs with the same price can be considered as indistinguishable, thus the induced symmetries can be broken by ordering lexicographically the corresponding columns.

$$p_k \geq p_{k+1}, \qquad \forall k = \overline{1, M-1}$$
$$p_k = p_{k+1} \Longrightarrow a_{\star k} \succ_{lex} a_{\star(k+1)}$$

Fixed values (FV). The search space can be reduced also by fixing the values of some variables starting from the application specific constraints. The strategy included in the experimental analysis is based on the exploitation of the conflict-type constraints as described in Section 4.2.2, i.e. the constraints given by (8)-(9) have been added to the problem specification.

Fixed values and price ordering (FVPR). This strategy uses FV to fix, on separate VMs, the conflicting components and PR to order the VMs. The list of VMs is not globally ordered but it is split in sublists which are ordered. This splitting is based on the structure of the clique with maximal deployment size ($\overline{G}$). More specifically, for each component in $\bar{G}$ the sublist containing the VMs on which its instances are deployed is decreasingly ordered based on price. Finally the VMs which do not contain instances of the components in $\bar{G}$ are decreasingly ordered by price (see Algorithm 1).

Fixed values and lexicographic ordering (FVLX). This strategy is similar to FVPR, the only difference being the fact that instead of imposing a price-based order on VMs it is applied a lexicographic order on the corresponding columns of the assignment matrix. More specifically, the lines 12 and 15 in Algorithm 1 are replaced with $CL.add(a_{\star h} \succ_{lex} a_{\star(h+1)} \succ_{lex} \ldots \succ_{lex} a_{\star(k-1)})$ and $CL.add(a_{\star k} \succ_{lex} a_{*(k+1)} \succ_{lex} \ldots \succ_{lex} a_{\star M})$, respectively.

Note that Algorithm 1 uses FV strategy, hence it requires the identification of all cliques in the conflict graph. At his aim, we used an implementation of Bron-Kerbosch algorithm available in NetworkX library.[15] Finding cliques in a graph is a NP-hard problem, however in our case studies the graph size (given by the number of components) is not very large, so this preprocessing step does not increase significantly the execution time.

It is worth mentioning that the strategies involved in the experimental analysis belong to several classes: *(i)* PR, LX and PRLX correspond to *column symmetry breakers* as they exploit full and partial symmetries in sets of columns corresponding to groups of VMs with similar characteristics (e.g. same price); *(ii)* FV is a *row symmetry breaker* exploiting the conflict-type constraints; *(iii)* FVPR and FVLX *combine column and row symmetry* by incorporating the advantages of the individual types.

[15] https://networkx.github.io.

Algorithm 1 FVPR algorithm.

1: Find the clique $\overline{G}$
2: $k \leftarrow 1$ /* k - VM index */
3: $CL = \emptyset$ /* CL - constraints list */
4: **for** each component $C_j \in \overline{G}$ **do**
5: $h \leftarrow k$
6: **for** each instance of C_j **do**
7: $CL.add(a_{jk} = 1)$
8: **for** each $t \in \{u \mid \mathcal{R}_{ju} = 1\}$ **do**
9: $CL.add(a_{tk} = 0)$
10: **end for**
11: $k \leftarrow k + 1$
12: $CL.add(p_h \geq p_{h+1} \geq \ldots \geq p_{k-1})$
13: **end for**
14: **end for**
15: $CL.add(p_k \geq p_{k+1} \geq \ldots \geq p_M)$
16: **return** CL

5.1.2. Software and hardware settings

In the case of the OMT solver Z3, as background theory, the formalization uses quantifier-free linear arithmetic, in particular quantifier-free linear integer arithmetic. This was chosen based on the results obtained in [27]. Z3 was used with the default values of the parameters.

In the case of CPLEX, besides the binary decision variables corresponding to the elements of the assignment matrix and the integer decision variables corresponding to the vector containing the types of VMs, the binary variables corresponding to the occupancy vector, v, have been explicitly included in the CP problem. This has been done in order to limit the number of variables generated by CPLEX for all constraints specified as logical expressions (as stated in Section 4.4.1). All CPLEX experiments have been conducted using primal reduction pre-processing (ensured by setting the option `parameters.preprocessing.reduce=1`), as the performance in this case was significantly better than in the other cases (no reduction or primal and dual reduction).

The implementation, as well as the results, are available at https://github.com/Maneuver-PED/RecommendationEngine. All reported timings are in seconds. They only include the actual solving time of the optimization problem and not the pre-processing steps like preparing the input and computing the initial numbers of VMs. In fact, the complexity of the pre-processing depends quadratically on the number of application components and number of VMs ($\mathcal{O}(max\{MN, N^2\})$).

All tests in this paper were performed on an Lenovo ThinkCentre with the following configuration: Intel® Core™ i7 vPro using CPLEX v12 and Z3 v4.8.7.

5.2. Results

Table 2 includes the results obtained without using symmetry breaking strategies. The estimated number of VMs reported on the second column has been obtained using the method described in Section 4.1. The list of offers was crawled from the Amazon site.[16] Each list of VM offers covers the main instance types, for example, `small`, `medium`, `large`. The list of offers can be viewed as a containment hierarchy (i.e. the list of 20 offers is included in the list of 40 offers etc.).

The tables include only those cases for which we obtained a result in a 40 minutes timeframe, as this was the time limit in the SMT-COMP'19.[17] The missing values (–) mean that no solution is returned in this timeframe.

One can observe that Z3 scales better than CPLEX, however, none of the tools scales for problems involving a large number of components' instances (e.g. at least 4 instances of the Wordpress component) and a large number of offers (more than a couple of dozens). This is due to the fact that the number O of VM offers influences the number of constraints generated, most notable $H \times O \times M$ constraints of type (3) and $M \times O$ constraints of type (4) are generated.

To overcome the lack of scalability issue, we applied the symmetry breaking strategies described in Section 5.1. The results are presented in Table 3.

5.3. Discussion

From the results reported in Table 3, we can draw the following remarks:

1. Using appropriate symmetry breakers and the OMT solver Z3 all problem instances were solved within the established timeframe. On the other hand, CPLEX shows lack of scalability, even if for small size problems it leads in a smaller amount of time to solutions (Fig. 6).

[16] https://aws.amazon.com/.
[17] https://smt-comp.github.io/2019/.

Table 2
Scalability tests for Z3 and CPLEX tools. Time values are expressed in seconds.

Problem	estimated #VMs	#offers=20		#offers=40		#offers=250		#offers=500	
		Z3	CPLEX	Z3	CPLEX	Z3	CPLEX	Z3	CPLEX
Oryx2	11	26.5	**0.16**	20.15	**0.42**	**365.77**	-	**881.34**	-
Sec. Web Container	6	**0.34**	5.68	**0.85**	101.84	**11.08**	-	**27.38**	-
Sec. Billing Email	5	0.21	**0.13**	**0.46**	3.62	**3.03**	324.8	**9.2**	-
Wordpress min#inst = 3	8	**2.26**	16.17	**7.54**	-	**223.66**	-	**547.2**	-
Wordpress min#inst = 4	10	50.37	**6.09**	**385.17**	-	-	-	-	-

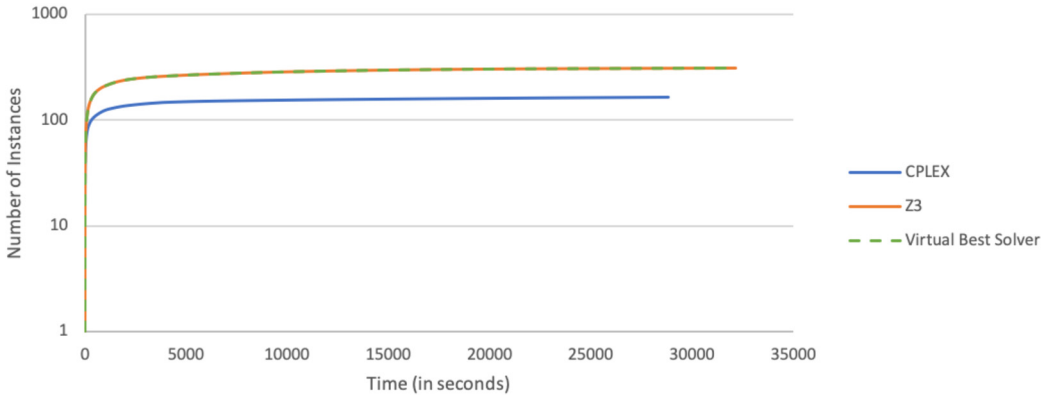


Fig. 6. Comparison between Z3 and CPLEX. (For interpretation of the colors in the figure, the reader is referred to the web version of this article.)

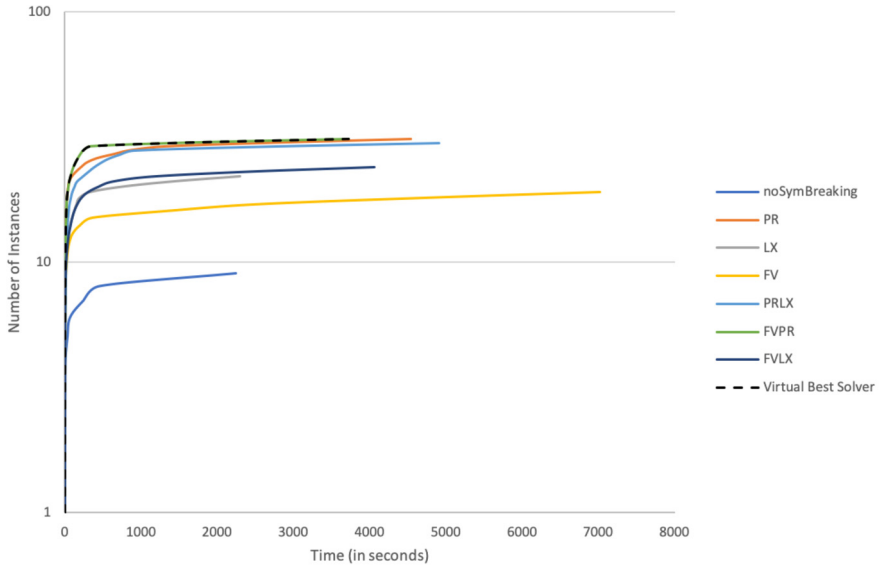


Fig. 7. Scalability of CPLEX for different symmetry breakers. (For interpretation of the colors in the figure, the reader is referred to the web version of this article.)

2. The best scalability is obtained by combining traditional column-wise symmetry breakers with search space reduction methods which exploit the graph representation associated to the structural constraints specific to each particular application (FVPR strategy) (Figs. 7 and 8).

Regarding the first remark, one possible explanation of the poorer scalability of CPLEX is that the number of variables increases significantly when the number of offers increases (see Table 4). This is due to the fact that in a preprocessing

Table 3
Efficiency analysis: time to find an optimal solution (in seconds) in case of applying different symmetry breaking strategies.

Problem	#offers=20						#offers=40						#offers=250						#offers=500					
	PR	LX	FV	PRLX	FVPR	FVLX	PR	LX	FV	PRLX	FVPR	FVLX	PR	LX	FV	PRLX	FVPR	FVLX	PR	LX	FV	PRLX	FVPR	FVLX
Z3 Solver																								
Oryx2	0.44	**0.33**	0.49	0.52	0.38	0.45	1.21	1.27	1.34	2.42	**1.04**	1.47	3.89	10.75	16.53	5.99	**2.34**	6.97	10.08	39.02	84.41	19.41	**9.83**	33.70
Sec. Billing Email	0.12	0.16	0.09	0.29	**0.08**	0.20	0.30	0.44	**0.27**	0.52	**0.27**	0.38	0.80	2.61	0.76	1.58	**0.68**	1.68	2.30	7.71	1.92	5.95	**1.85**	2.68
Sec. Web Container	0.13	0.16	0.12	0.21	**0.11**	0.20	0.41	0.60	**0.37**	0.62	0.39	0.53	0.99	3.35	**1.52**	1.65	1.54	3.86	3.24	6.70	**2.37**	4.40	3.48	4.69
Wordpress min#inst=3	0.27	0.32	0.31	0.27	**0.23**	0.44	**0.69**	1.15	1.09	1.18	0.71	1.29	**1.36**	7.10	12.57	5.00	2.46	6.13	4.27	29.45	24.36	10.88	**8.05**	25.49
Wordpress min#inst=4	0.41	0.59	0.58	0.61	**0.35**	0.61	1.34	2.09	2.73	1.83	**1.11**	2.63	3.92	13.65	49.84	6.64	**3.63**	14.89	12.26	54.50	161.00	21.33	**9.15**	37.99
Wordpress min#inst=5	0.60	0.88	1.11	0.71	**0.48**	1.06	2.23	2.98	5.84	2.61	**1.63**	2.91	12.17	18.12	167.88	8.55	**7.26**	21.46	22.86	77.51	787.07	33.53	**15.25**	72.55
Wordpress min#inst=6	0.82	1.17	1.40	1.25	**0.49**	1.16	2.40	4.50	6.11	3.11	**1.70**	4.39	11.37	24.25	824.55	15.98	**6.36**	45.78	25.96	117.47	-	45.50	**17.31**	59.71
Wordpress min#inst=7	1.15	1.68	2.32	1.36	**0.75**	1.46	4.78	6.05	19.30	4.49	**2.47**	7.76	23.93	40.70	-	27.24	**8.30**	43.81	46.05	140.93	-	55.32	**28.13**	92.17
Wordpress min#inst=8	1.95	2.56	15.25	2.23	**0.99**	2.56	15.91	13.06	174.09	6.60	**4.47**	15.45	139.67	107.48	-	71.43	**28.00**	161.76	260.44	302.54	-	415.67	**79.64**	303.21
Wordpress min#inst=9	1.97	3.34	23.96	2.63	**1.06**	3.10	15.62	16.50	202.34	8.00	**4.66**	17.69	273.28	111.81	-	139.91	**33.64**	167.59	375.29	396.45	-	421.62	**87.92**	253.93
Wordpress min#inst=10	5.95	4.30	71.71	3.24	**1.50**	5.41	62.44	21.93	1008.30	12.79	**6.06**	26.24	492.09	126.14	-	127.68	**34.33**	170.95	-	467.52	-	1187.05	**240.36**	476.49
Wordpress min#inst=11	27.24	5.45	253.62	3.46	**1.91**	5.97	216.74	28.96	2328.61	24.74	**6.76**	41.91	2078.36	155.49	-	414.06	**70.77**	239.72	-	436.16	-	-	**338.74**	671.25
Wordpress min#inst=12	28.43	5.55	239.09	4.67	**1.92**	7.14	225.30	32.69	-	17.64	**8.41**	33.03	1755.92	188.86	-	414.70	**33.32**	245.59	-	690.17	-	-	**418.80**	623.72
CPLEX Solver																								
Oryx2	0.05	14.06	0.20	1.36	**0.03**	1.23	0.14	68.22	0.19	5.45	**0.06**	12.35	5.25	-	60.08	94.82	**3.04**	925.00	247.11	-	-	-	**27.30**	-
Sec. Billing Email	0.07	1.32	0.05	0.38	**0.06**	0.11	0.35	5.03	0.23	1.48	**0.21**	0.42	58.56	-	29.22	234.97	**31.98**	37.89	-	-	-	-	**376.39**	-
Sec. Web Container	0.31	10.53	0.15	2.40	0.12	**0.10**	3.60	142.85	0.33	12.78	**0.30**	0.31	347.47	-	74.70	-	**68.86**	58.08	-	-	-	-	**1316.53**	1795.03
Wordpress min#inst=3	**0.19**	85.58	0.89	0.72	0.24	2.37	**1.55**	-	42.77	7.88	1.97	39.62	**346.77**	-	-	1588.69	1582.11	-	-	-	-	-	-	-
Wordpress min#inst=4	**0.32**	910.36	1.84	0.84	**0.32**	4.93	4.48	-	-	10.27	**3.63**	153.73	**1252.09**	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=5	**0.50**	-	4.05	4.18	0.54	18.63	7.04	-	-	45.68	**3.58**	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=6	**0.67**	-	14.73	5.21	0.68	18.49	14.03	-	-	36.24	**8.76**	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=7	**0.39**	-	12.05	7.75	1.92	27.22	29.01	-	-	124.74	**25.04**	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=8	**0.54**	-	1554.52	7.53	1.77	38.41	30.07	-	-	141.97	**23.68**	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=9	**0.77**	-	25.57	7.46	2.40	39.10	**17.16**	-	-	73.49	26.84	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=10	3.75	-	1908.48	9.57	**1.87**	58.52	74.72	-	-	168.08	**43.22**	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=11	**2.55**	-	98.84	24.54	2.87	109.82	96.82	-	-	320.81	**29.20**	-	-	-	-	-	-	-	-	-	-	-	-	-
Wordpress min#inst=12	4.70	-	-	13.97	**2.29**	63.03	239.28	-	-	580.61	**44.26**	-	-	-	-	-	-	-	-	-	-	-	-	-

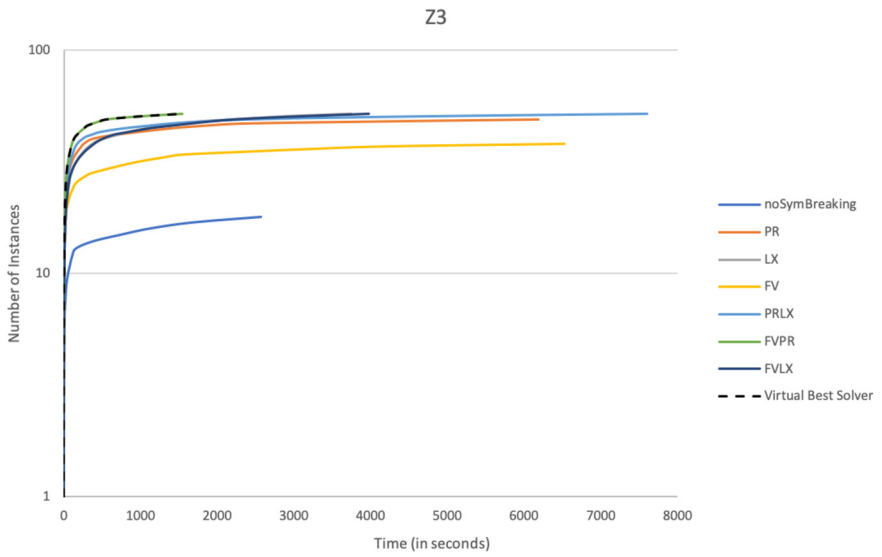


Fig. 8. Scalability of Z3 for different symmetry breakers. (For interpretation of the colors in the figure, the reader is referred to the web version of this article.)

Table 4
Number and types of variables used by CPLEX (explicit decision variables and auxiliary generated variables) - no symmetry breaking case.

Problem	Binary	Integer	Total	Binary	Integer	Total	Binary	Integer	Total	Binary	Integer	Total
	#offers=20			#offers=40			#offers=250			#offers=500		
Oryx2	387	63	**450**	644	63	**707**	3339	63	**3402**	6547	63	**6610**
Sec. Billing Email	148	29	**177**	265	29	**294**	1490	29	**1519**	2948	29	**2977**
Sec. Web Container	178	34	**212**	318	34	**352**	1788	34	**1822**	3538	34	**3572**
Wordpress min#inst=3	242	46	**288**	429	46	**475**	2389	46	**2435**	4722	46	**4768**
Wordpress min#inst=4	302	57	**359**	535	57	**592**	2985	57	**3042**	5902	57	**5959**
Wordpress min#inst=5	362	69	**431**	642	69	**711**	3582	69	**3651**	7082	69	**7151**
Wordpress min#inst=6	392	75	**467**	695	75	**770**	3880	75	**3955**	7672	75	**7747**
Wordpress min#inst=7	452	86	**538**	802	86	**888**	4477	86	**4563**	8852	86	**8938**
Wordpress min#inst=8	511	98	**609**	908	98	**1006**	5073	98	**5171**	10031	98	**10129**
Wordpress min#inst=9	541	103	**644**	961	103	**1064**	5371	103	**5474**	10621	103	**10724**
Wordpress min#inst=10	601	115	**716**	1068	115	**1183**	5968	115	**6083**	11801	115	**11916**
Wordpress min#inst=11	661	126	**787**	1174	126	**1300**	6564	126	**6690**	13014	126	**13140**
Wordpress min#inst=12	691	132	**823**	1227	132	**1359**	6862	132	**6994**	13621	132	**13753**

step, CPLEX translates the original problem into a MP formulation, in particular, all logical implications are rewritten and auxiliary variables are introduced.

Regarding the second remark, first we notice that, in the case when Z3 solver is used, out of the single-criterion strategies, the LX one leads to significantly better results than PR and FV strategies. This is particularly true in the case of large size problems (i.e. many components and many offers, as is the case of Wordpress). However, since LX strategy leads to many implication type constraints (12) this has a negative impact in the case of CPLEX solver.

We observe that the FV strategy scales worse than the other individual strategies in the case of the Wordpress application with a large number of offers and a high number of Wordpress instances. On the other hand, as it is expected, FV strategy is effective for problems for which the number of pairwise conflicting components is large and a significant reduction in the number of variables can be obtained (e.g. Secure Billing Email, Secure Web Container - see Table 5). In these cases combining FV with other strategies (e.g. FVPR, FVLX) does not bring a benefit because the search space exhibits none or

Table 5
Problems characteristics and ratio of fixed decision variables: Number of fixed values as determined by FV strategy (columns 1-3); Estimated vs. used number of VMs (columns 4-5); Number of components vs. number of deployed instances (columns 6-7).

Problem	#fixed values of a	Total#vars of a	Ratio	estimated #VMs	#occupied VMs	#components	#deployed instances
Oryx2	6	110	5%	11	6	10	26
Sec. Web Container	18	30	60%	6	5	5	8
Sec. Billing Email	12	25	48%	5	5	6	6
Wordpress min#inst=3	9	40	22%	8	8	5	8
Wordpress min#inst=4	12	50	24%	10	10	5	10
Wordpress min#inst=5	15	60	25%	12	12	5	12
Wordpress min#inst=6	18	65	27%	13	13	5	13
Wordpress min#inst=7	21	75	28%	15	15	5	15
Wordpress min#inst=8	24	85	28%	17	17	5	17
Wordpress min#inst=9	27	90	30%	18	18	5	18
Wordpress min#inst=10	30	100	30%	20	20	5	20
Wordpress min#inst=11	33	110	30%	22	22	5	22
Wordpress min#inst=12	36	115	31%	23	23	5	23

very few symmetries after applying FV and the additional symmetry breaking constraints only increases the computational burden.

As column-wise symmetry breaker, the PR strategy has the disadvantage of not being able to break all symmetries (as the VMs used in the deployment might have the same price) but the advantage of leading to simpler, thus less computationally costly, symmetry breaking constraints. Therefore, in the case of CPLEX solver the PR strategy leads to better results than LX strategy. On the other hand, in the case of Z3 solver, LX scales better than PR because it turns out that only few types of VMs are typically used for deployment, hence ordering based on price does not break too many symmetries.

Out of the three combined strategies (PRLX, FVPR, FVLX), the one leading consistently to the best results is FVPR, especially in the case of large size problem instances. Even in the cases when CPLEX solver has not reached a solution in the established timeframe for PR and FV, the combined strategy has obtained a good performance. This is because fixing values to variables is equivalent to their elimination.

Although one would expect that FVLX outperforms PRLX as variables are fixed, this is the case only for Wordpress with at least 8 deployed instances. In fact, the benefit of combining FV and LX, even when compared with LX strategy, can be observed only for the problem instances mentioned above.

6. Related work

The work presented in this paper was performed in the framework of the project *MANeUveR: MANagement agency for cloUd Resources*, which sought to answer questions like: *Which CPs offer the best infrastructure at a fair budget? I am no Cloud expert then what are the characteristics of the infrastructure which best fit my application?* At the heart of MANeUveR lies a *Recommendation Engine* module which proposes a *deployment plan* fulfilling application (interactions between components, their hardware and software specifications) and user (e.g. optimal price) requirements. It is based on some preliminary results obtained in the European-funded projects MODAClouds[18] and SPECS[19] and the national-funded project AMICAS[20] [28,29,19], namely: *(i)* the idea of a Recommendation Engine came from MODAClouds [28], since their decision support system did not use automatic matching between user requirements and services characteristics, a feature which was necessary for application deployment in a multi-cloud environment; *(ii)* the work on automated acquiring and configuring cloud resources based on Security Service Level Agreements performed in SPECS motivated us to study the scalability of existing optimization methods used to enhance certain performance indicators [19]; *(iii)* in the AMICAS project, we presented some preliminary experiments on the scalability of a newly developed method combining exact and heuristic methods for *resource management*

[18] http://www.modaclouds.eu/.
[19] http://www.specs-project.eu/.
[20] https://amicas.hpc.uvt.ro/.

problems but in a simpler context: only conflict type constraints were considered and the price minimization was not addressed [29].

Compared to [19], *(i)* our formalization considers different hardware characteristics of both components and VMs offers as well extensive types of interaction constraints between application components; *(ii)* we do not fix the initial number of VMs needed for deployment but we approximate it for each application and then refine it to an optimum; *(iii)* we also perform scalability tests using a realistic number of VMs offers crawled from various Cloud Providers. Paper [19] is extended by [30] where the authors consider also hardware characteristics. Distinctly to our approach, they model the deployment problem as a double assignment problem, which links components to instances (i.e. types of VMs offers) and instances to VMs offers. Distinctly to our methods for optimal deployment (delivering a single optimal solution) their optimization process aims to identify the deployment configuration that fulfills the existing performance and security constraints while ensuring the best compromise between the deployment cost and the overall level of security achieved, based on a scale of preferences expressed by the developer. This is achieved by using the AHP methodology. Also, no scalability tests of their method is performed.

For delivering the deployment plan, in the *Recommendation Engine* we implemented approximate (population-based metaheuristics) and exact methods. As the focus of this paper is on exact approaches, we focus on *mathematical programming* and *SMT/OMT* methods enhanced with symmetry breaking techniques applied to problem formalizations similar to ours.

Mathematical programming is heavily used for the improvement of Cloud resource management (see [31] for a survey). Some of them use symmetry breaking for speeding-up the solution process. As an example, [32] formalizes the so-called temporal bin packing problem, that is assignment of a set of tasks (items) to a set of machines (bins) under capacity constraints (CPU usage) where items have a lifespan such that the cost of using the bins is minimized. The problem is solved from two perspectives: Mixed Integer Programming (CPLEX[21]) and Constraint Programming (Gecode[22]). The problem approached in [32] is also a generalization of bin packing but, unlike the class of problems we address, it does not involve interactions (e.g. conflicts) between tasks. The only symmetry breakers used are those imposing an order over the time for which bins are allocated or on the bins usage.

The usage of *SMT/OMT techniques* in Cloud deployment problems is scarce. Related work includes [21], where the authors solve the following problem: given a high-level specification of the desired system (the set of available components together with their requirements i.e. constraints between components) and the maximal amount of VMs that can be leased, place the components on the VMs such that the minimum number of VMs is used and automated deployment of the system in the cloud environment is achieved. At this aim, they construct a toolchain composed of Zephyrus, for the planning phase, and Armonic, for the deployment phase. Relevant for our work is Zephyrus, since its input is translated into a set of constraints over non-negative integers (in the MiniZinc[23] constraint modeling language) and uses various constraint programming solvers for finding the solution. Different to our approach, they use a predefined number of VM types with known hardware specifications based on the prior knowledge of the application requirements. Zephyrus can be proven correct and complete: it will always find a configuration that is optimal. Paper [21] is extended by [17]. The problem solved is that, given the application description (interaction constraints and hardware requirements) and VMs specifications, the aim is to minimize the cost and then the number of VMs leased for hosting the application. They achieve this by using CP solvers (Gecode,[24] Google OR-Tools,[25] Chuffed[26]) and SMT solvers (Z3). The paper presents various comparisons on how the tools perform on different types of constraints (linear/nonlinear), the conclusion being that the SMT solvers perform better for nonlinear constraints while are being outperformed by CP in the linear case. Regarding the similarities/dissimilarities to our approach, we mention the following. *(i)* Zephyrus2 solves deployment optimization problems by translating them into a COP encoded in MiniZinc. By default, it solves the resulting multi-objective optimization problems by optimizing the first objective function value (cost) and then optimizing the other objective function (number of components) sequentially following their order after substituting the previously determined optimal values. This solution has the drawback that the solver has to be restarted. Differently, we have a single optimization problem (the optimization criteria being the cost) since, until now, in our use cases we did not encounter situations when more components than required by users are assigned to VMs. *(ii)* In Zephyrus2, the list of offers is limited (only four types of VMs); in the MANeUveR approach this list is dynamically constructed based on the cloud providers offers. *(iii)* In Zephyrus2, according to the provided examples, only memory and price requirements are specified in the problem description; however their approach can be easily extended to other type constraints. However, no performance information are available for more complex problem formulations.

SMT solvers were applied also to other types of NP-hard problems, similar to ours. For example, [33] solves the Flexible Job-Shop Scheduling Problem: we are given a number of jobs, each job being defined by a sequence of operations. Each operation can be executed by any machine from a given pool of machines for a predefined execution time. A machine can be used by multiple operations but only one operation is allowed to execute on a single machine at a time. The task

[21] https://www.ibm.com/analytics/cplex-optimizer.
[22] https://www.gecode.org.
[23] http://www.minizinc.org.
[24] https://www.gecode.org.
[25] https://developers.google.com/optimization.
[26] https://github.com/chuffed/chuffed.

is to assign operations to machines such that the total time to complete all jobs is minimized, i.e. minimization of the makespan. The paper proposes three formulations of the problem which are handled by Z3 SMT solver. The experimental results revealed the fact that the SMT solver does not scale well for all formulations. As a remark, the constraints involved in the formulations have arithmetic simpler than ours, while the number of machines used is rather small (at most 7).

Regarding the literature on *symmetry breaking techniques* relevant for our problem, there are two categories of papers: *(i)* experimental papers using symmetry breaking techniques to speed-up the solution process but which do not apply a methodology in their usage (e.g. [17,32]); *(ii)* theoretical papers developing symmetry breaking techniques for different versions of the bin-packing problem but which are not directly applicable to our problem (e.g. [34,23,35–38]). What makes our problem challenging from the theoretical point of view is the presence of various types of structural constraints. For example, [34] develops a theory for symmetry breaking for job scheduling problems characterized by the fact that a job can be assigned to a single machine from a pool of different machines and there are not multiple instances of the same job.[27] This leads to a problem where all symmetries can be eliminated. They attempt to extend it to machine scheduling problem,[28] which is closer to our problem. However, in this case it is not possible to break all symmetries, hence they also propose an experimental approach. Differently to us, they used dynamic symmetry breaking technique (orbital branching) which proved to be more effective than static symmetry breaking when integrated in CPLEX. It worth noticing that the machine problem they consider is much simpler than ours. In the theoretical framework developed by [23], they study how lexicographic ordering the rows/columns of a matrix model, respectively their composition influences the symmetry breaking. They applied it to the problem of balanced incomplete block design generation, that is an arrangement of n distinct objects into b blocks, such that each block contains exactly d distinct objects, each object occurring in exactly r different blocks, and every two distinct objects occur together in exactly λ blocks. The experimental results reveal the fact that the column symmetries are more efficient than the row ones, while lexicographically ordering the rows and columns can break most of the compositions of the row and column symmetries. Note that the problem constraints are simpler than ours. Paper [35] does not consider any optimality fulfillment criteria except the minimal number of bins; same paper proposes various symmetry breaking methods whose evaluation is proposed; we did not find any continuation of their work. Paper [36] considers a problem similar to ours, namely the rack configuration problem, proposes two formalizations of it each exhibiting various column symmetries, which are systematically described. Differently to our approach, in order to break them, they propose the combination of these formalizations. The problem is simpler than ours as no interactions between the card inserted in the racks are present. The experimental results were performed in CONJUNTO [37], a tool different to MP and SMT/OMT as it uses interval reasoning on finite sets. [38] studies the efficiency of different symmetry breaking techniques and their combination in the framework of job grouping problems, i.e. the task of assigning a set of jobs, each with a specific set of tool requirements, to machines with a limited tool capacity in order to minimize the number of machines needed. This problem is simpler than ours as: *(i)* it minimizes the number of machines used and not the price, *(ii)* the requirement of each job is constant, *(iii)* the machines are identical, and *(iv)* there is no dependency/interaction between jobs. Symmetry determined by the identical machines is eliminated by reformulating the problem. In our case, this is handled by the symmetry breakers (10), (11) and their valid combinations. Except, problem reformulation, they propose other symmetry breakers, for example variable reduction and lexicographic ordering constraints, both on rows and columns. Distinctly from our approach, which reduces the number of variables by fixing values to certain variables from a, their meaning of variable reduction is that the assignment of a job i to a higher indexed machine j is not allowed. The techniques are tested on an academic dataset using CPLEX with symmetry breaking option activated.

7. Conclusions

We proposed several strategies to tackle the scalability issues in the case of optimal deployment of component-based applications in the Cloud. On one hand, this issue was addressed by observing that problem symmetry is the result of the possible presence of identical virtual machines, which is propagated at various levels, e.g. the way components are assigned to VMs. On the other hand, the particularities of each application are exploited, in particular the symmetries determined by the cliques of the components being in conflict.

The main conclusion of the experimental analysis is that by combining simple symmetry breaking strategies, one can obtain an effective one. More specifically, by combining a variable reduction strategy with a column-wise symmetry breaker, the scalability was achieved for all problem instances when using the OMT solver Z3. However, CPLEX proved to be appropriate only for small size problems (those involving a couple of dozens of VM offers). One reason for that is that the formulation of the Cloud deployment problem is general and does not exploit the particularities of the CPLEX solver. Hence, for better scalability of CPLEX, the Cloud deployment problem from Section 3 should be reformulated.

As future work we plan to assess the effectiveness of symmetry breaking sorting criteria in designed greedy approaches for updating cloud deployment plans.

[27] In terms of our problem, this is equivalent to have in matrix a on each row precisely one 1, the rows are not constraints dependent and the columns can not be assigned the same type.

[28] In terms of our problem, this is equivalent to have in matrix a no restrictions on the number of 1 in each row.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation, 1st edition, Addison-Wesley Professional, 2010.
[2] N. Bjørner, SpecOps: Infusing Azure with Verifiable Reliability (Invited Talk), Formal Foundations of Software Defined Networks (held in conjunction with the VI IEEE Conference on Network Function Virtualization and Software Defined Networks), 2020.
[3] C.C. Din, R. Bubel, R. Hähnle, KeY-ABS: a deductive verification tool for the concurrent modelling language ABS, in: A.P. Felty, A. Middeldorp (Eds.), Automated Deduction - CADE-25, Springer International Publishing, Cham, 2015, pp. 517–526.
[4] E. Albert, P. Arenas, A. Flores-Montoya, S. Genaim, M. Gómez-Zamalloa, E. Martin-Martin, G. Puebla, G. Román-Díez, SACO: static analyzer for concurrent objects, in: E. Ábrahám, K. Havelund (Eds.), Tools and Algorithms for the Construction and Analysis of Systems, Springer, Berlin, Heidelberg, 2014, pp. 562–567.
[5] B. Burns, J. Beda, K. Hightower (Eds.), Kubernetes: Up and Running Dive into the Future of Infrastructure, O'Reilly, 2019.
[6] S. Dutta, S. Gera, A. Verma, B. Viswanathan, SmartScale: automatic application scaling in enterprise clouds, in: 2012 IEEE Fifth International Conference on Cloud Computing, 2012, pp. 221–228.
[7] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang, L. Zaruvinsky, M. Espinosa, R. Lin, Z. Liu, J. Padilla, C. Delimitrou, An open-source benchmark suite for microservices and their hardware-software implications for Cloud & Edge systems, in: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '19, Association for Computing Machinery, New York, NY, USA, 2019, pp. 3–18.
[8] S. Martello, D. Pisinger, D. Vigo, E.D. Boef, J. Korst, Algorithm 864: general and robot-packable variants of the three-dimensional bin packing problem, ACM Trans. Math. Softw. 33 (1) (2007), 7–es.
[9] E.G. Coffman Jr., M.R. Garey, D.S. Johnson, An application of bin-packing to multiprocessor scheduling, SIAM J. Comput. 7 (1) (1978) 1–17.
[10] W. Fernandez de la Vega, G.S. Lueker, Bin-packing can be solved within $1 + \epsilon$ in linear time, Combinatorica 1 (4) (1981) 349–355.
[11] A. Beloglazov, R. Buyya, Energy efficient resource management in virtualized cloud data centers, in: Proceedings of the 2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing, CCGRID '10, IEEE Computer Society, Washington, DC, USA, 2010, pp. 826–831.
[12] J.M.V. de Carvalho, LP models for bin-packing and cutting stock problems, Eur. J. Oper. Res. 141 (2) (2002) 253–273.
[13] B. Korte, J. Vygen, Combinatorial Optimization: Theory and Algorithms, 5th edition, Springer Publishing Company, Incorporated, 2012.
[14] B. Jennings, R. Stadler, Resource management in clouds: survey and research challenges, J. Netw. Syst. Manag. 23 (3) (2015) 567–619.
[15] IBM, IBM ILOG IBM ILOG CPLEX Optimization Studio CPLEX User's Manual. Version 12, 2016.
[16] N. Bjørner, A. Phan, L. Fleckenstein, νZ - an optimizing SMT solver, in: Tools and Algorithms for the Construction and Analysis of Systems - 21st International Conference, TACAS 2015, London, UK, April 11-18, 2015. Proceedings, 2015, pp. 194–199.
[17] E. Ábrahám, F. Corzilius, E.B. Johnsen, G. Kremer, J. Mauro, Zephyrus2: on the fly deployment optimization using SMT and CP technologies, in: Dependable Software Engineering: Theories, Tools, and Applications - Second International Symposium, SETTA 2016, Beijing, China, November 9-11, 2016, Proceedings, 2016, pp. 229–245.
[18] L. de Moura, N. Bjørner, Z3: an efficient SMT solver, in: C.R. Ramakrishnan, J. Rehof (Eds.), Tools and Algorithms for the Construction and Analysis of Systems, Springer, Berlin, Heidelberg, 2008, pp. 337–340.
[19] V. Casola, A.D. Benedictis, M. Erascu, J. Modic, M. Rak, Automatically enforcing security SLAs in the cloud, IEEE Trans. Serv. Comput. 10 (5) (2017) 741–755.
[20] M. Erascu, G. Iuhasz, F. Micota, An architecture for a management agency for cloud resources, in: 20th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing, SYNASC 2018, Timisoara, Romania, September 20-23, 2018, 2018, pp. 288–295.
[21] R.D. Cosmo, M. Lienhardt, R. Treinen, S. Zacchiroli, J. Zwolakowski, A. Eiche, A. Agahi, Automated synthesis and deployment of cloud applications, in: ACM/IEEE International Conference on Automated Software Engineering, ASE '14, Vasteras, Sweden - September 15 - 19, 2014, 2014, pp. 211–222.
[22] P. Flener, A. Frisch, B. Hnich, Z. Kiziltan, I. Miguel, T. Walsh, Matrix modelling: exploiting common patterns in constraint programming, in: A. Frisch (Ed.), Proceedings of the International Workshop on Reformulating Constraint Satisfaction Problems, 2002, pp. 27–41.
[23] P. Flener, A.M. Frisch, B. Hnich, Z. Kiziltan, I. Miguel, J. Pearson, T. Walsh, Breaking row and column symmetries in matrix models, in: Principles and Practice of Constraint Programming - CP 2002, 8th International Conference, CP 2002, Ithaca, NY, USA, September 9-13, 2002, Proceedings, 2002, pp. 462–476.
[24] F. Rossi, P. van Beek, T. Walsh (Eds.), Handbook of Constraint Programming, Foundations of Artificial Intelligence, vol. 2, Elsevier, 2006.
[25] R. Sebastiani, P. Trentin, OptiMathSAT: a tool for optimization modulo theories, in: D. Kroening, C.S. Păsăreanu (Eds.), Computer Aided Verification, Springer International Publishing, Cham, 2015, pp. 447–454.
[26] Y. Li, A. Albarghouthi, Z. Kincaid, A. Gurfinkel, M. Chechik, Symbolic optimization with SMT solvers, in: The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL '14, San Diego, CA, USA, January 20–21, 2014, 2014, pp. 607–618.
[27] M. Erascu, R. Metes, Constrained optimization benchmark for optimization modulo theory: a cloud resource management problem, in: SMT 2019 17th International Workshop on Satisfiability Modulo Theories, Affiliated with SAT, July 7-8, 2019, Lisbon, Portugal, 2019, pp. 7–8.
[28] S. Gupta, J. Dominiak, P. Matthews, V. Muntés Mulero, A. Omerovic, Decision making toolkit prototype – final version, Tech. rep., 2015.
[29] M. Eraşcu, F. Micota, D. Zaharie, A scalable hybrid approach for applications placement in the cloud, in: 2015 Conference Grid, Cloud High Performance Computing in Science, ROLCG, 2015, pp. 1–4.
[30] V. Casola, A.D. Benedictis, M. Rak, U. Villano, Security-by-design in multi-cloud applications: an optimization approach, Inf. Sci. 454–455 (2018) 344–362.
[31] J. Zhang, H. Huang, X. Wang, Resource provision algorithms in Cloud Computing: a survey, J. Netw. Comput. Appl. 64 (2016) 23–42.
[32] M. De Cauwer, D. Mehta, B. O'Sullivan, The temporal bin-packing problem: an application to workload management in data centres, in: 2016 IEEE 28th International Conference on Tools with Artificial Intelligence, ICTAI, 2016, pp. 157–164.
[33] S.F. Roselli, K. Bengtsson, K. Akesson, SMT solvers for flexible job-shop scheduling problems: a computational analysis, in: 2019 IEEE 15th International Conference on Automation Science and Engineering, CASE, 2019.
[34] M.F. Anjos, J.P. Ostrowski, A. Vannelli, Symmetry in Scheduling Problems, 2010.
[35] J. Régin, M. Rezgui, Discussion about constraint programming bin-packing models, in: AI for Data Center Management and Cloud Computing, Papers from the 2011 AAAI Workshop, San Francisco, California, USA, August 7, 2011, 2011.
[36] Z. Kiziltan, B. Hnich, Symmetry breaking in a rack configuration problem, in: Proceedings of the IJCAI 2001 Workshop on Modelling and Solving Problems with Constraints, 2001.
[37] C. Gervet, Interval propagation to reason about sets: definition and implementation of a practical language, Constraints 1 (3) (1997) 191–244.
[38] R. Jans, J. Desrosiers, Efficient symmetry breaking formulations for the job grouping problem, Comput. Oper. Res. 40 (4) (2013) 1132–1142.